合肥工業大學

2024~2025 学年 第二学期

《系统硬件综合设计》

设计报告

班 级 计科 22-1 班

学 号 2022212119

姓 名 李易简

2024 年 12 月

目 录

1 设计要求 3
2.流水线架构设计 3
2.1 各阶段功能总览 4
2.2 每个阶段详细功能以及流水传递 4
2.3 流水设计思路 7
3 主要模块详解 8
3.1 PC 模块 8
3.2 INS 模块 9
3.3 分支预测器 9
3.4 ID 模块 10
3.5 CU 模块 11
3.6 寄存器堆 13
3.7 数据前推模块 14
3.8 冲突判断模块 15
3.9 EXE 模块 16
3.10 MEM 模块 19
3.11 RAM 模块 21
3.12 WB 模块 22
4 RISCV 指令格式 23
4.1 指令格式分析 23
4.2 指令实现汇总 23
4.3 指令实现验证 24
5 数据前推相关问题 28
5.1 数据前推相关问题以及实现 28
5.2 数据前推上版测试 29
5.3 数据前推优化分析 31
6 Load 冒险相关问题 32
6.1 Load 冒险相关问题以及实现 32
6.2 冒险相关问题上板演示 33
7 分支预测相关问题以及实现 35
7.1 分支预测原理及其实现 35
7.2 分支预测失败解决方案 36
7.3 分支预测失败现象上板 36
7.4 分支预测优化分析 38
8 总结 39

1 设计要求

基于先修课程，根据系统设计思想，使用硬件描述语言设计实现一款基于 MIPS32，ARM，RISC-V 或者自定义指令集的微处理器（CPU）。要求：完成单周期 CPU 设计，或多周期 CPU 设计，或多级流水线 CPU 设计（递进式、难度依次提升。所有学生必须至少完成单周期 CPU 的设计工作），可以将设计的 CPU 下载至 FPGA 开发板（ego-1）上运行。以此贯穿数字逻辑、计算机组成原理、计算机体系结构课程，实现从逻辑门至完整 CPU 处理器的设计。

1. **团队设计完成情况：**实现 RISCV 指令架构下的多周期五级流水线 CPU，乘除法器，实现数据前推，解决关于 Load 的数据冒险以及控制冒险，同时实现 2bit 配合全局历史表进行动态预测，实现指令总数 41。

2. **个人设计完成情况：**设计 RISCV 指令 R：10、I：7、B：5、S：3，总共 23 条指令。设计了多周期整体框架以及五级流水线之间的流水处理，并实现了数据前推，冲突判断模块，能够解决 Load 后的指令冲突以及控制冒险，同时设计并实现 2bit 配合全局历史表的动态分支预测模块。

2. 流水线架构设计

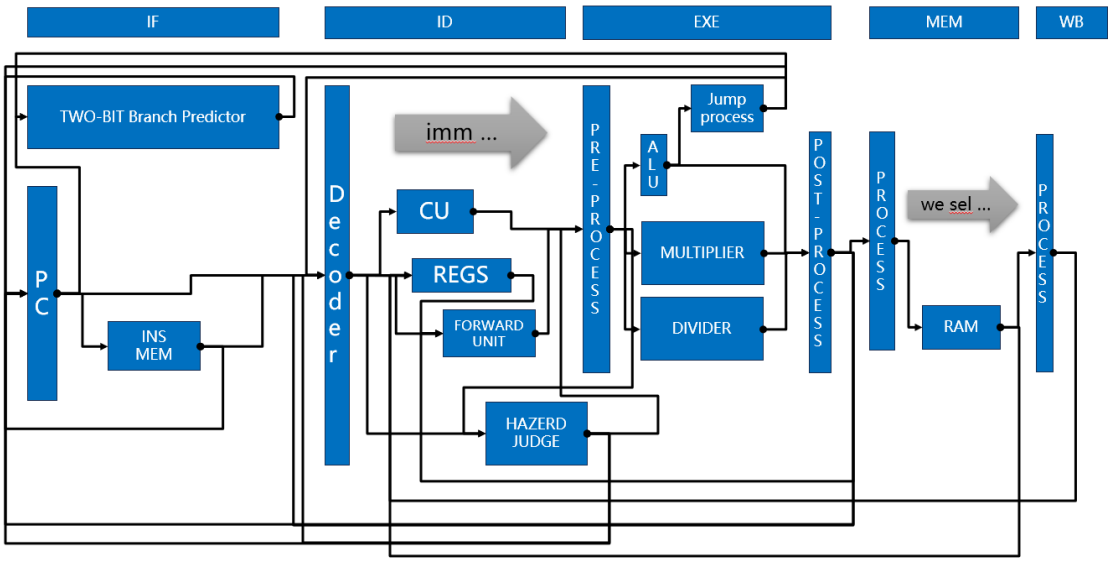


图 2.1 抽象模块连通图

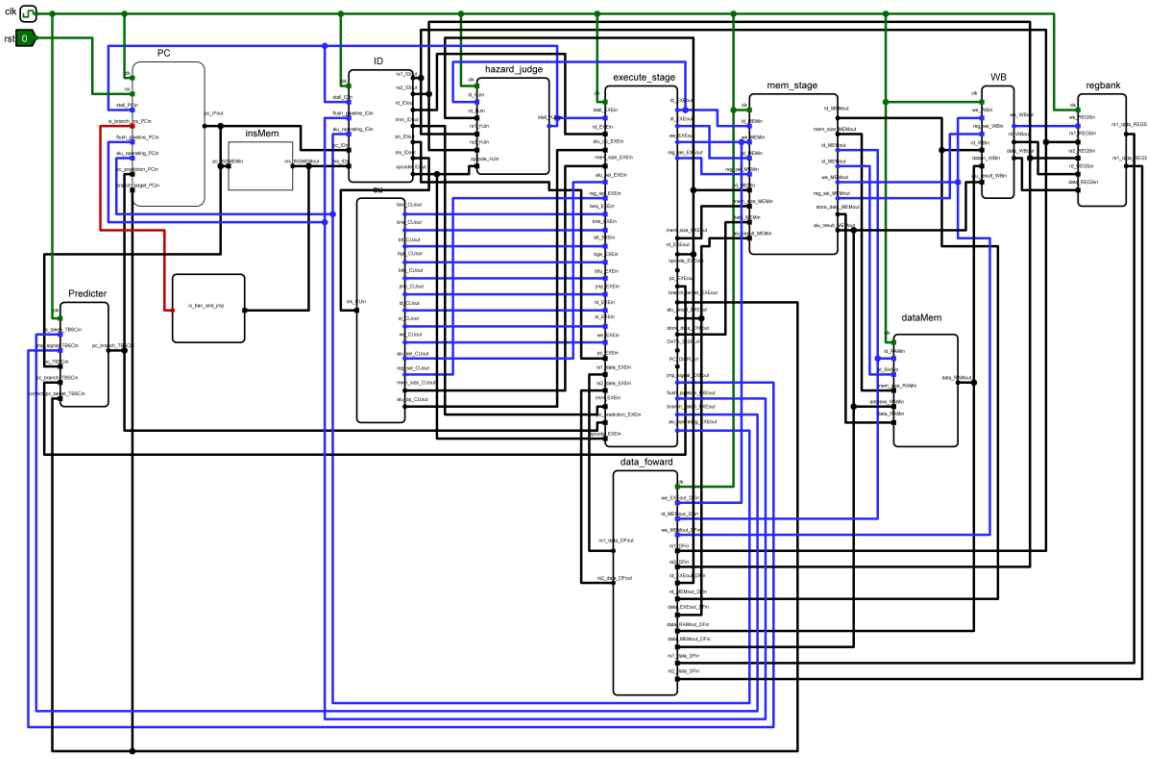


图 2.2 顶层数据流通图

2.1 各阶段功能总览

IF 阶段，实现通过 PC 值进而去指令单元地址获取对应指令并更新 PC 值。

ID 阶段，通过对 IF 阶段传出的 INS 进行解码处理，并从寄存器获取数据。

EXE 阶段，通过 CU 解码出来的信号对从 ID 阶段输入的数据进行计算。

MEM 阶段，从 RAM 中取出或者存入数据。

WB 阶段，对寄存器进行回写。

2.2 每个阶段详细功能以及流水传递

a) IF 阶段首先通过 PC 模块，获取指令地址，然后从指令内存中取出对应指令，并将该指令是否为分支指令的信息回传给 PC 模块； PC 模块同时也会将 PC 值传递至 2bit 预测器，预测器通过传入 pc 值结合过往历史跳转表进行分支预测，并将预测地址重新传给 PC 模块，PC 模块在下个上升沿发生时根据预测地址取出指令完成分支预测。

IF 阶段并将 PC 值以及 INS 作为该阶段的输出流向 ID 阶段。

2bit 预测器模块同时需要接受来自 EXE 阶段的信号输出，以便更新对应地址的分支指令的历史跳转表以及 2bit 状态值。

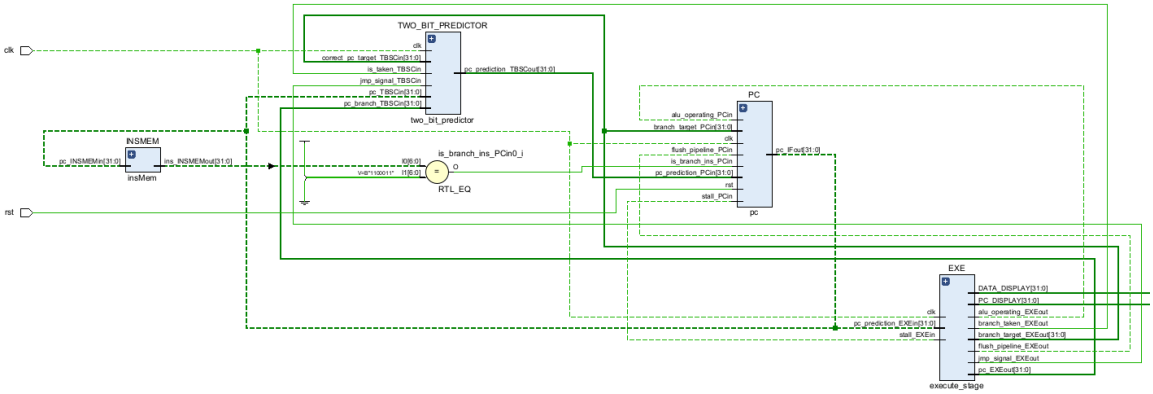


图 2.2.1 IF 阶段数据流通图

b) ID 阶段，通过接受来自 IF 阶段的流水数据，需要对指令进行解码，在 decoder 模块中负责对指令中的数据进行预处理，包括提取立即数、符号拓展，以及提取两个源寄存器的地址和目标寄存器的地址等需求。

同时将 pc 等数据正常向下传递。

将两个源寄存器传递给 REGS 中获取对应源寄存器地址的数据，寄存器堆将两个数据传出，送至数据前推模块。

同时指令流入 CU，控制单元根据指令类型给各种使能信号进行赋值并流出。

数据前推模块，接受来自 REGS 流出的数据，以及 decoder 模块流出的两个源操作寄存器地址，并接受来自 EXE 阶段的回写信号以及计算结果，同时还有 MEM 阶段的回写信号、计算结果以及访存信号、访存数据，将这些数据前推至数据前推模块完成数据冒险问题的解决，并将最新的寄存器数据作为输出传递给 EXE 模块作为 ALU 的计算输入。

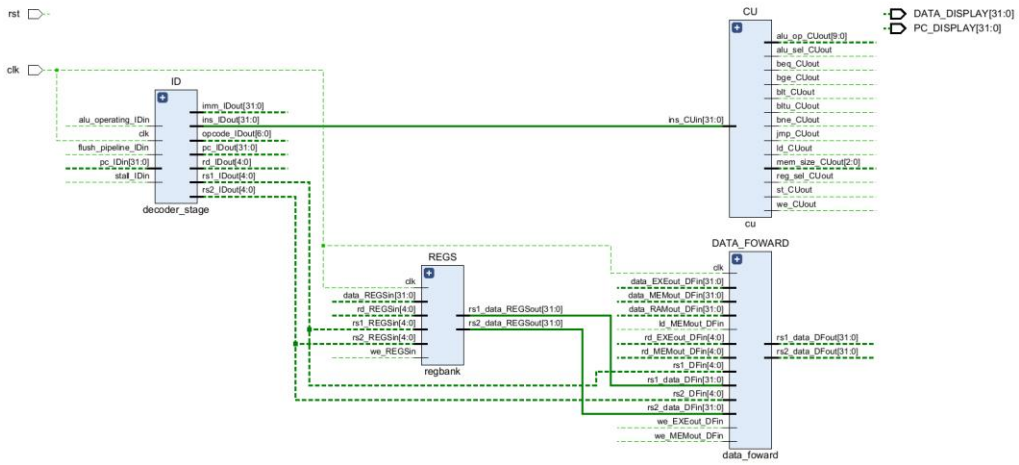


图 2.2.2 ID 阶段数据流通图

c) EXE 阶段，接受来自 ID 阶段个计算数据以及各种使能信号，根据不同类型的指令完成对应的计算操作，以及数据正常流动。

假如为普通逻辑指令，那么根据输入数据进行逻辑计算即可。

假如为分支指令，计算是否跳转条件以及提取分支指令地址，并向前传递以满足可能存在的冲刷流水线的需求以及更新预测器。

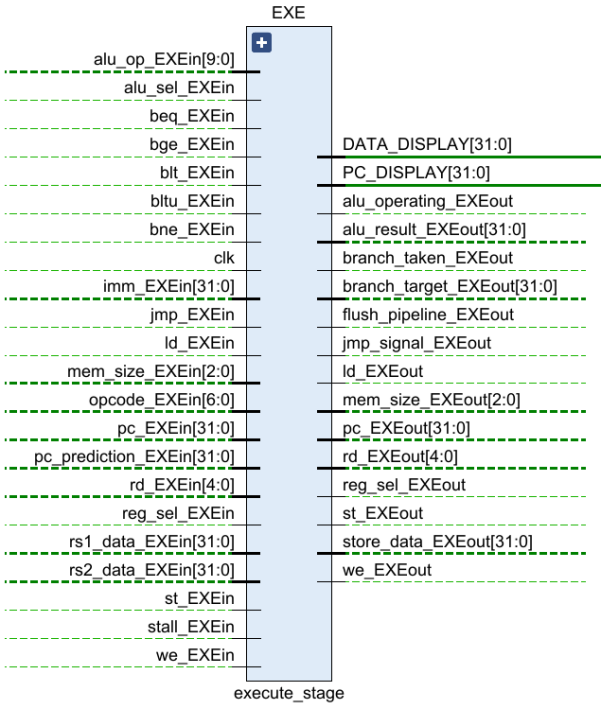


图 2.2.3 EXE 阶段数据流通图

d) MEM 阶段，根据使能信号判断是否为访存指令，倘若 ld 使能或者 st 使能为真，根据不同访存指令大小，前往对应地址不同大小地址块进行取数或者存数，并同时将一些数据进行流水传递。

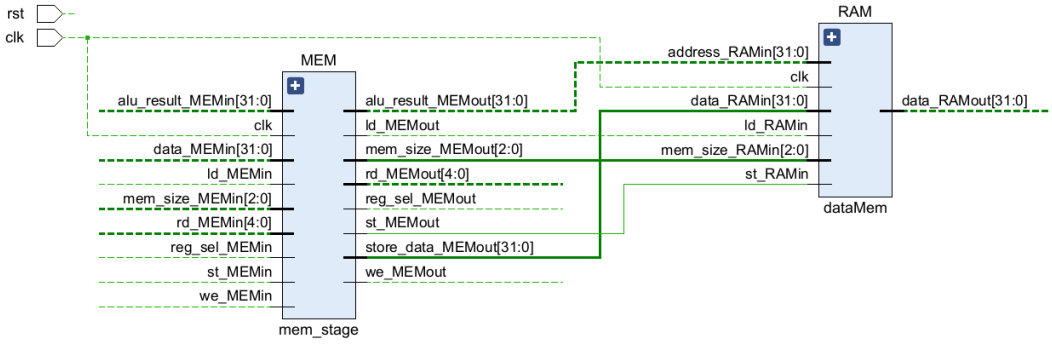


图 2.2.4 MEM 阶段数据流通图

e) WB 阶段，根据 wb 回写使能信号以及回写数据，向寄存器组传递回写数据，将回写数据进行回传。

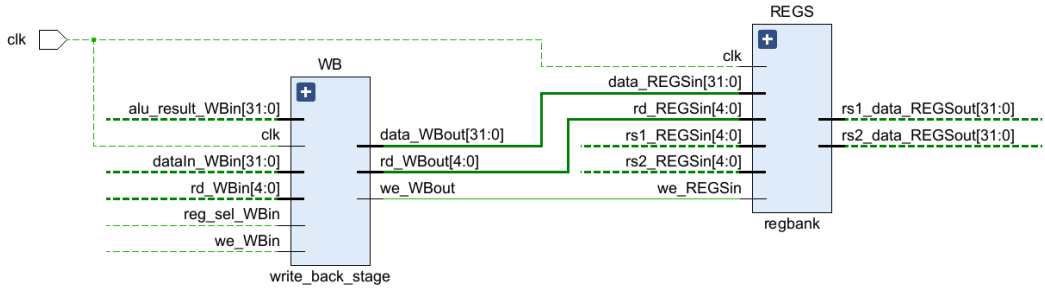


图 2.2.5 WB 阶段数据流通图

2.3 流水设计思路

每个阶段在上升沿便能获取从上个阶段准备好的数据，在上升沿，倘若该数据本阶段不需要，仅作为流水数据那么直接向下一级传递即可，若该阶段需要该数据参与计算，那么我们需要将该数据缓存到模块内部的寄存器中（因为输入数据作为上个阶段的输出数据，会跟随上个阶段的数据处理改变，以及前两个阶段会有潜在的阻塞的要求，因此需要将相关数据暂做缓存例如 IF 阶段的 pc 值以及 ID 阶段的 INS 值），在该阶段完成数据处理的任务后便可以将数据向下一层模块进行传递，更新到对应的线性输入。

3 主要模块详解

3.1 PC 模块

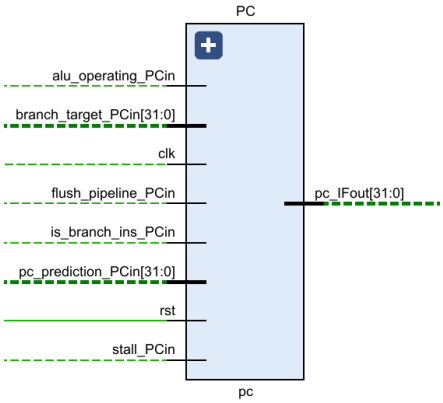


图 3.1 pc 模块数据流

数据	描述	output/input
clk	时钟信号	input
rst	重置信号	input
stall_PCin	阻塞冲突信号输入	input
alu_operating_PCin	乘除法器工作信号输入	input
is_branch_ins_PCin	上一条指令是否为分支指令输入	input
branch_target_PCin	正确分支指令 PC 值输入	input
flush_pipeline_PCin	流水线冲刷信号输入	input
pc_prediction_PCin	分支预测地址输入	input
pc_IFout	当前指令 PC 值输出	output

3.2 INS 模块

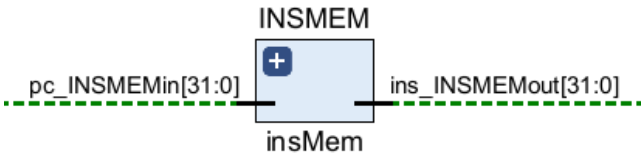


图 3.2 INSMEM 模块数据流

数据	描述	output/input
pc_INSMEMin	指令地址输入	input
ins_INSMEMout	指令输出	output

3.3 分支预测器

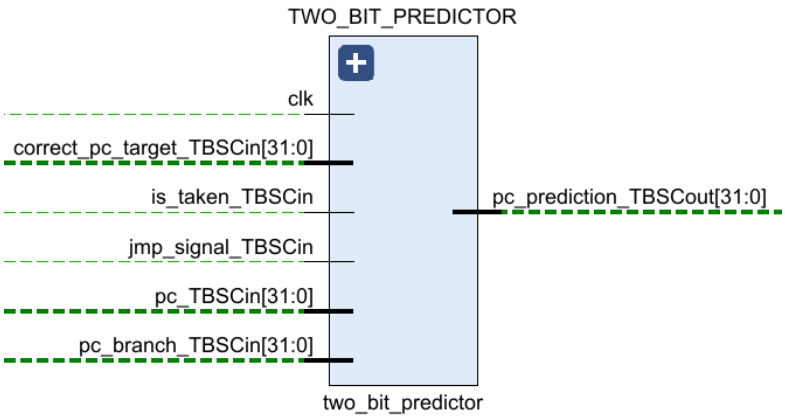


图 3.3 分支预测器模块数据流

数据	描述	output/input
clk	时钟信号	input
is_taken_TBSCin	EXE 阶段分支指令是否跳转	input

jmp_signal_TBSCin	EXE 阶段是否为分支指令	input
pc_TBSCin	IF 阶段分支指令的 PC 值	input
correct_pc_target_TBSCin	EXE 阶段分支指令正确跳转地址	input
pc_branch_TBSCin	EXE 阶段分支指令 PC 值	input
pc_prediction_TBSCout	分支预测器预测跳转地址	output

3.4 ID 模块

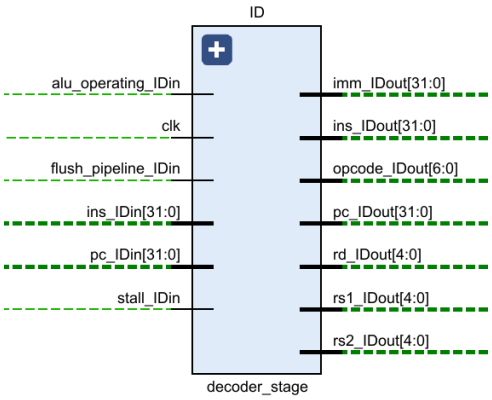


图 3.4 ID 模块数据流

数据	描述	output/input
clk	时钟信号	input
stall_IDin,	冲突阻塞信号输入	input
flush_pipeline_IDin	流水线冲刷信号输入	input
alu_operating_IDin	乘除法器工作信号输入	input

pc_IDin	指令 pc 值输入	input
ins_IDin	指令输入	input
rs1_IDout,	源寄存器地址一输出	output
rs2_IDout,	源寄存器地址二输出	output
rd_IDout,	目标地址寄存器输出	output
imm_IDout,	立即数输出	output
pc_IDout,	该阶段指令 pc 输出	output
ins_IDout,	该阶段指令输出	output
opcode_IDout	指令类型码输出	output

3.5 CU 模块

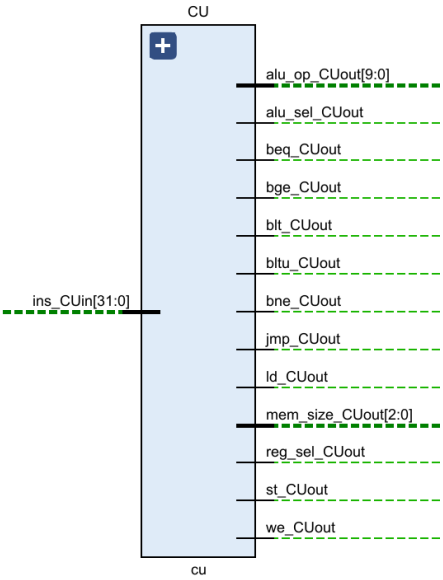


图 3.5 控制单元模块数据流

数据	描述	output/input
ins_CUin	指令输入输入	input
beq_CUout	beq 分支指令使能输出	output
bne_CUout	bne 分支指令使能输出	output
blt_CUout	blt 分支指令使能输出	output
bge_CUout	bge 分支指令使能输出	output
bltu_CUout	bltu 分支指令使能输出	output
jmp_CUout	jmp 指令使能输出	output
ld_CUout	访存指令使能输出	output
st_CUout	存数指令使能输出	output
we_CUout	回写使能输出	output
alu_sel_CUout	alu 选择使能输出	output
reg_sel_CUout	WB 阶段回写数据选择使能输出	output
mem_size_CUout	访存地址大小输出	output
alu_op_CUout	alu 运算码输出	output

3.6 寄存器堆

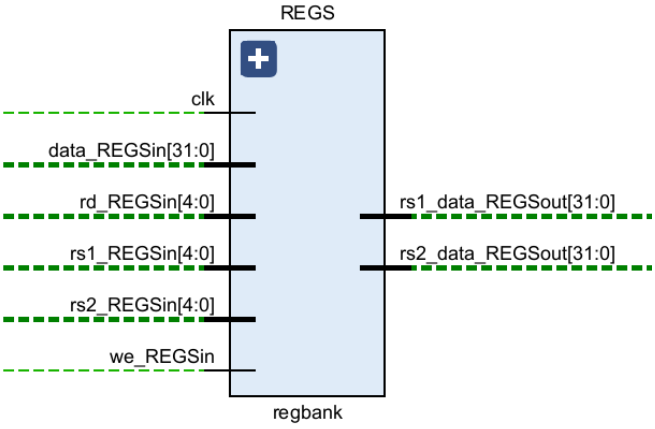


图 3.6 寄存器堆模块数据流

数据	描述	output/input
clk	时钟信号	input
we_REGSin	回写使能输入	input
rs1_REGSin	读出源寄存器地址一输入	input
rs2_REGSin	读出源寄存器地址二输入	input
rd_REGSin	回写寄存器地址输入	input
data_REGSin	回写数据输入	input
rs1_data_REGSout	寄存器地址一读出数据	output
rs2_data_REGSout	寄存器地址二读出数据	output

3.7 数据前推模块

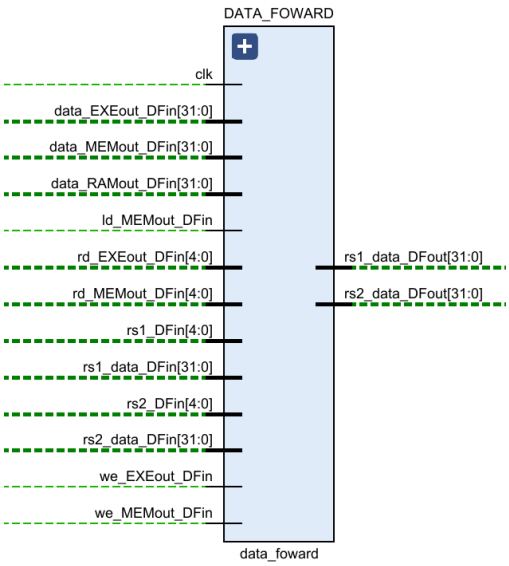


图 3.7 数据前推模块数据流

数据	描述	output/input
clk	时钟信号	input
we_EXEout_DFin	EXE 阶段回写使能输入	input
ld_MEMout_DFin	MEM 阶段访存使能输入	input
we_MEMout_DFin	MEM 阶段回写使能输入	input
rs1_DFin	ID 阶段源寄存器地址一输入	input
rs2_DFin	ID 阶段源寄存器地址二输入	input
rd_EXEout_DFin	EXE 阶段回写寄存器地址输入	input
rd_MEMout_DFin	MEM 阶段回写寄存器地址输入	input

data_EXEout_DFin	EXE 阶段回写数据输入	input
data_RAMout_DFin	RAM 读出数据输入	input
data_MEMout_DFin	MEM 阶段回写数据输入	input
rs1_data_DFin	寄存器读出源寄存器数据一输入	input
rs2_data_DFin	寄存器读出源寄存器数据二输入	input
rs1_data_DFout	数据前推寄存器一输出数据	output
rs2_data_DFout	数据前推寄存器二输出数据	output

3.8 冲突判断模块

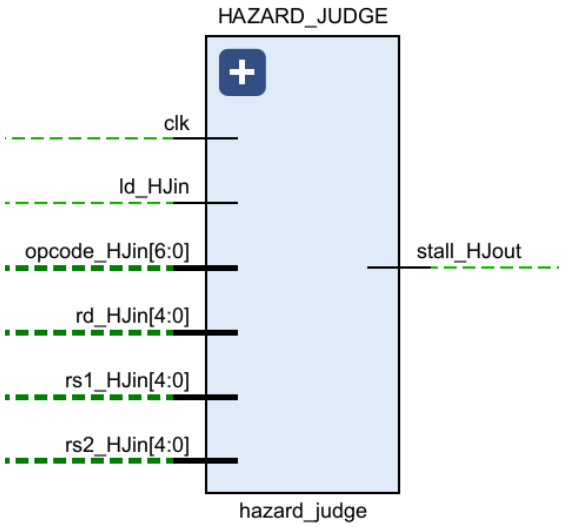


图 3.8 冲突控制检测模块数据流

数据	描述	output/input
clk	时钟信号	input

ld_HJin	EXE 阶段访存使能输入	input
rd_HJin	EXE 阶段目标寄存器地址输入	input
rs1_HJin	ID 阶段源寄存器地址一输入	input
rs2_HJin	ID 阶段源寄存器地址二输入	input
opcode_HJin	ID 阶段指令类型码输入	input
stall_HJout	冲突阻塞信号	output

3.9 EXE 模块

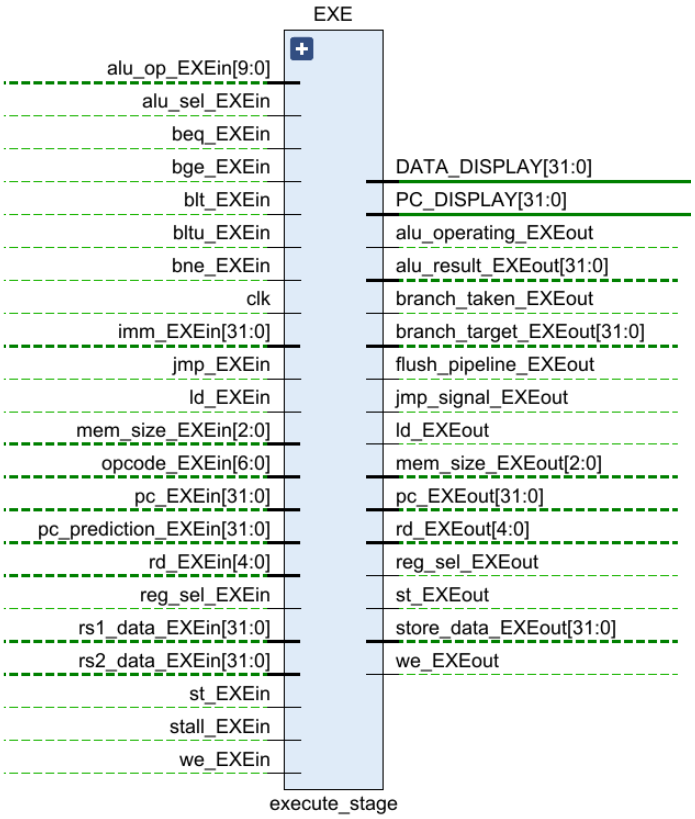


图 3.9 EXE 模块数据流

数据	描述	output/input
clk	时钟信号	input
stall_EXEin	冲突阻塞信号输入	input
rd_EXEin	目标寄存器地址输入	input
alu_op_EXEin	alu 运算码输入	input
mem_size_EXEin	访存大小输入	input
alu_sel_EXEin	alu 选择使能输入	input
reg_sel_EXEin	wb 阶段选择使能输入	input
beq_EXEin	beq 分支指令使能输入	input
bne_EXEin	bne 分支指令使能输入	input
blt_EXEin	blt 分支指令使能输入	input
bge_EXEin	bge 分支指令使能输入	input
bltu_EXEin	bltu 分支指令使能输入	input
jmp_EXEin	jmp 指令使能输入	input
ld_EXEin	访存使能输入	input
st_EXEin	存数使能输入	input
we_EXEin	回写使能输入	input

pc_EXEin	ID 阶段 pc 输入	input
rs1_data_EXEin	输入计算数据一输入	input
rs2_data_EXEin	输入计算数据二输入	input
imm_EXEin	立即数输入	input
pc_prediction_EXEin,	分支指令预测地址输入	input
opcode_EXEin	指令种类码输入	input
rd_EXEout	写入目标寄存器输出	output
mem_size_EXEout	访存大小输出	output
pc_EXEout	当前阶段 pc 值输出	output
branch_target_EXEout	分支指令正确跳转地址输出	output
alu_result_EXEout	alu 运算结果输出	output
store_data_EXEout	储存输出	output
DATA_DISPLAY	上板数据显示输出	output
PC_DISPLAY	上班 pc 显示输出	output
ld_EXEout	访存使能输出	output
st_EXEout	存数使能输出	output
we_EXEout	回写使能输出	output

reg_sel_EXEout	wb 阶段选择使能输出	output
jmp_signal_EXEout	分支指令信号输出	output
flush_pipeline_EXEout	冲刷流水线信号输出	output
branch_taken_EXEout	分支跳转信号输出	output
alu_operating_EXEout	乘除法器工作信号输出	output
opcode_EXEout	该阶段指令种类码输出	output

3.10 MEM 模块

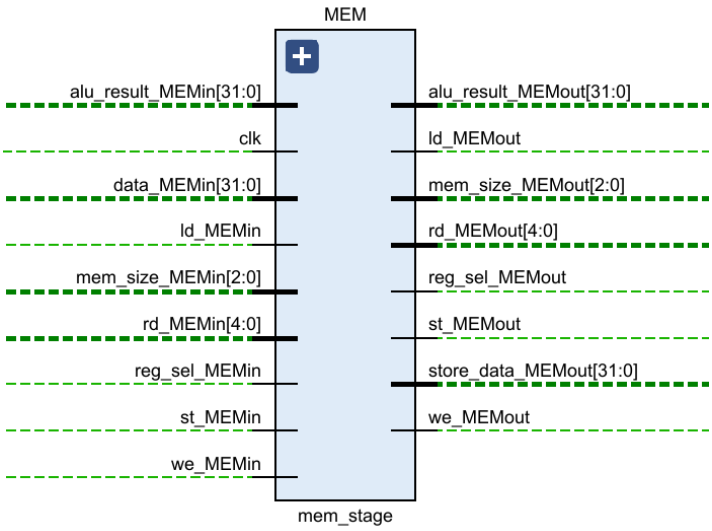


图 3.10 MEM 模块数据流

数据	描述	output/input
clk	时钟信号	input
ld_MEMin	访存使能输入	input

st_MEMin	存数使能输入	input
we_MEMin	回写使能输入	input
reg_sel_MEMin	wb 阶段选择使能输入	input
rd_MEMin	目标寄存器地址输入	input
mem_size_MEMin	访存大小输入	input
data_MEMin	存数数据输入	input
alu_result_MEMin	alu 计算结果输入	input
rd_MEMout	目标寄存器地址输出	output
mem_size_MEMout	访存大小输出	output
ld_MEMout	访存使能输出	output
st_MEMout	存数使能输出	output
we_MEMout	回写使能输出	output
reg_sel_MEMout	wb 阶段选择使能输出	output
store_data_MEMout	alu 运算结果输出	output
alu_result_MEMout	储存输出	output

3.11 RAM 模块

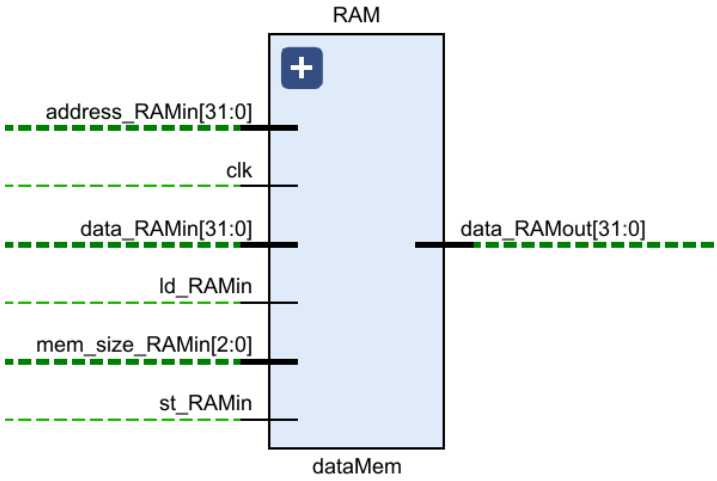


图 3.11 RAM 模块数据流

数据	描述	output/input
clk	时钟信号	input
ld_RAMin	访存信号使能输入	input
st_RAMin	存数信号使能输入	input
mem_size_RAMin	访存大小输入	input
address_RAMin	访存地址输入	input
data_RAMin	储存数据输入	input
data_RAMout	访存数据结果输出	output

3.12 WB 模块

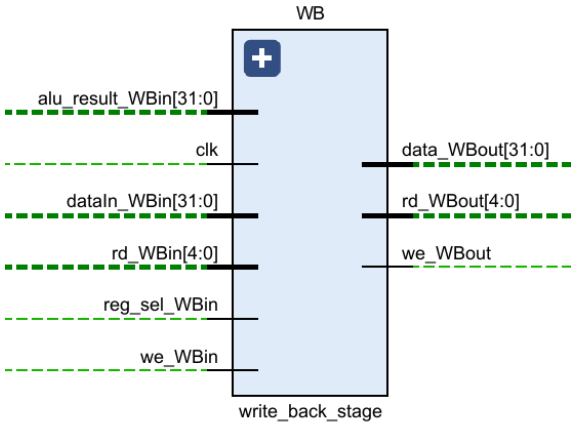


图 3.12 WB 模块数据流

数据	描述	output/input
clk	时钟信号	input
we_WBin	回写使能输入	input
reg_sel_WBin	wb 阶段选择使能输入	input
rd_WBin	目标寄存器地址输入	input
dataIn_WBin	回写数据输入	input
alu_result_WBin	alu 计算结果输入	input
we_WBout	回写使能输出	output
rd_WBout	目标寄存器回写地址输出	output
data_WBout	回写数据输出	output

4 RISCV 指令格式

4.1 指令格式分析

31 27 26 25	24 20	19 15	14 12	11 7	6 0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]				rd	opcode	U-type
imm[20|10:1|11|19:12]				rd	opcode	J-type

图 4.1 RISCV 指令格式

指令参数解释：

funct7、funct 指令操作码；opcode 表明指令类型； imm 立即数

rs1：源寄存器地址一 ；rs2：源寄存器地址二；rd：目标寄存器地址

4.2指令实现汇总

表 4.2.1 团队实现指令汇总

R-TYPE	I-TYPE	B-TYPE	S-TYPE	J-TYPE	伪指令
ADD	ADDI	BEQ	SB	JAL	NOP
AND	ANDI	BGE	SH	JARL	NOT
SUB	ORI	BLT	SW		NEG
OR	XORI	BNE			MV
SLL	LB	BLTU			LI
SRA	LH				J
SRL	LW				JR
XOR	LUI				
DIV	SRLI				
MUL	SRAI				
SLT	SLLI				
REM	AUIPC				

表 4.2.2 个人实现指令汇总

R-TYPE	I-TYPE	B-TYPE	S-TYPE
ADD	ADDI	BEQ	SB
AND	ANDI	BGE	SH
SUB	ORI	BLT	SW
OR	XORI	BNE	
SLL	LB	BLTU	
SRA	LH		
SRL	LW		
XOR			

4.3指令实现验证

```
ins_test1.dat
00000000 00100000 10000010 10110011 // add x5, x1, x2
00000000 00010010 11110011 00110011 // and x6, x5, x1
01000000 00100011 00000011 10110011 // sub x7, x6, x2
00000000 00100000 11100100 00110011 // or  x8, x1, x2
00000000 00100011 10010100 10110011 // sll x9, x7, x2
01000000 00100011 11010100 00110011 // sra x8, x7, x2
00000000 00100011 11010011 10110011 // srl x7, x7, x2
00000000 10010011 11000011 00110011 // xor x6, x7, x9
```

图 4.3.1 第一组测试指令–逻辑指令

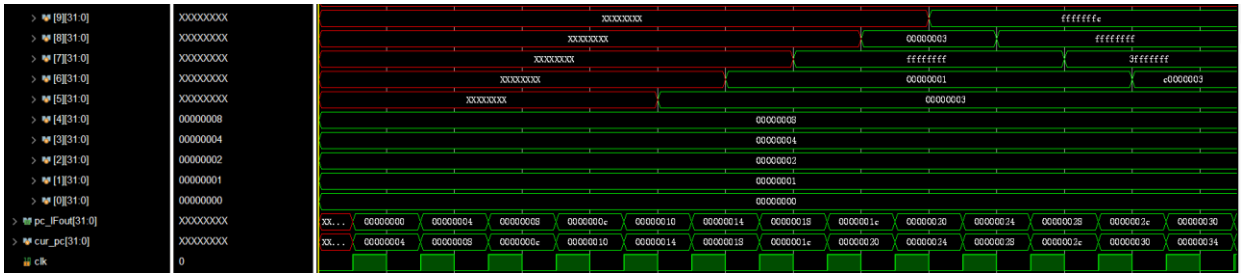


图 4.3.2 模拟波形

第五个周期的下降沿 REGS 内部数值开始发生改变，按照指令顺序依次更新为指令结果。

add x5, x1, x2；　（1+2）= 3；

将 x1 寄存器值与 x2 寄存器值相加，结果存至 x5。

and x6, x5, x1；　（3 & 1）= 1

将 x5 寄存器值与 x1 寄存器值相与，结果存至 x6。

sub x7, x6, x2；　（1-2）= ffffffff

将 x6 寄存器值与 x2 寄存器值相减，结果存至 x7。

or x8, x1, x2；　（1 | 2）= 3

将 x1 寄存器值与 x2 寄存器值相或，结果存至 x8。

sll x9, x7, x2；　（ffffffff << 2）= fffffffe

将 x7 寄存器值左移 x2 寄存器值，结果存至 x9

sra x8, x7, x2；　(ffffffff >>> 2) = ffffffff

将 x7 寄存器值算数右移 x2 寄存器值相加，结果存至 x8

srl x7, x7, x2；　(ffffffff >> 2) = effffffff

将 x7 寄存器值逻辑右移 x2 寄存器值相加，结果存至 x7

xor x6, x7, x9；　(effffffff ^ fffffffe) = 3000000e

将 x7 寄存器值与 x9 寄存器值异或，结果存至 x6

ins_test2.dat

```
00000000 00010001 00000010 10010011 // addi x5, x2, 1
00000000 00110001 01110011 00010011 // andi x6, x2, 3
00000000 01010010 11100011 10010011 // ori  x7, x5, 5
00000000 01110010 11000100 00010011 // xori x8, x5, 7
00000000 01000000 00000011 10000011 // lb   x7, 4(x0)
00000000 01000000 00010011 10000011 // lh   x7, 4(x0)
00000000 01000000 00100011 10000011 // lw   x7, 4(x0)
00000000 01110000 00000000 00100011 // sb   x7, 0(x0)
00000000 01110000 00010000 00100011 // sh   x7, 0(x0)
00000000 01110000 00100000 00100011 // sw   x7, 0(x0)
```

图 4.3.3 I-TYPE 和 S-TYPE 指令测试

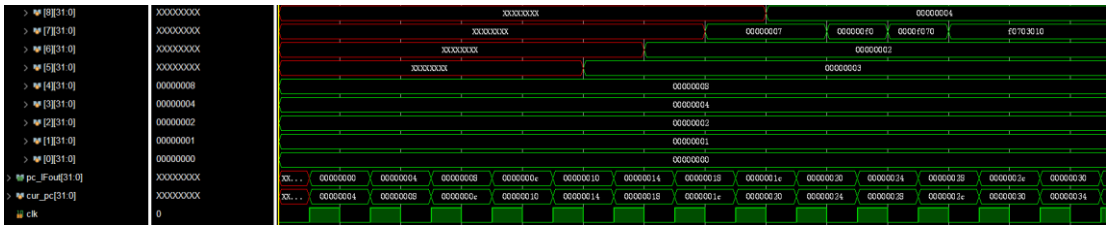


图 4.3.4 REGS 状态更新显示

第五个周期的下降沿 REGS 数值开始发生改变，按照指令顺序依次更新结果。

addi x5, x2, 1；　（2+1）= 3；　将 x1 寄存器值与 1 相加，结果存至 x5。

andi x6, x2, 3；　（2 & 3）= 3；　将 x2 寄存器值与 3 相与，结果存至 x6。

ori x7, x5, 5；　（3 | 5）= 7；　将 x5 寄存器值与 5 相或，结果存至 x7。

xori x8, x5, 7；　（3 ^ 7）= 4；　将 x5 寄存器值与 7 异或，结果存至 x8。

表 4.3.1 初始 RAM 内存数据

单元地址块	初始储存数据	单元地址块	初始储存数据
0000	0F	0004	F0
0001	07	0005	70
0002	03	0006	30
0003	01	0007	10

lb　x7, 4(x0);　将地址块为 4 开始地址加载一个字节到 x7 （F0）

lh　x7, 4(x0);　将地址块为 4 开始地址加载一个半字到 x7（F070）

lw　x7, 4(x0);　将地址块为 4 开始地址加载一个整字到 x7（F0703010）

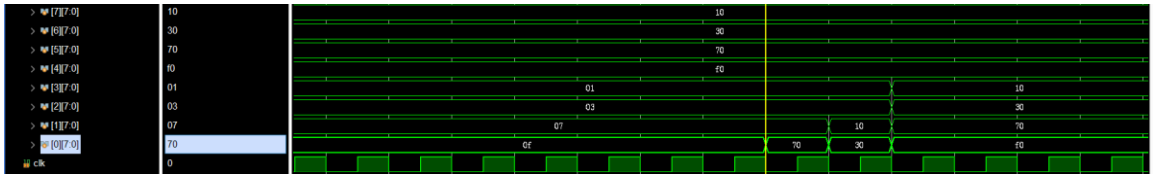


图 4.3.5 RAM 状态更新显示

在最后一个 lw 指令回写的同时，sb 指令完成内存数据的储存。

将 x7 中的数据依照字节、半字以及整字的数据大小依次周期储存进地址下标为 0 的空间块中。（x7 = F0703010）

sb　x7, 0(x0)；　将 x7 的低位字节加载到开始地址为 0 开始的一个字节空间中。

sh　x7, 0(x0)；　将 x7 的低位字节加载到开始地址为 0 开始的一个半字空间中。

sw　x7, 0(x0)；　将 x7 的低位字节加载到开始地址为 0 开始的一个整字空间中。

```
12      reg [31:0] regs [31:0];
13
14      initial begin
15          regs[0] <= 32'b0;
16
17          regs[1] <= 32'b0001;
18          regs[2] <= 32'b0010;
19          regs[3] <= 32'b0100;
20          regs[4] <= 32'b1000;
21      end
```

图 4. 3. 6 寄存器初始状态

```
ins_test3.dat
1   00000000 00010001 00000110 01100011 // beq x2, x1, flag1
2   00000000 00010000 10000100 01100011 // beq x1, x1, flag1
3   00000000 00000000 00000000 00010011 // nop
4   00000000 00100000 11010110 01100011 // bge x1, x2, flag2    (flag1)
5   00000000 00010001 01010100 01100011 // bge x2, x1, flag2
6   00000000 00000000 00000000 00010011 // nop
7   00000000 00100001 01000110 01100011 // blt x2, x2, flag3    (flag2)
8   00000000 00100000 11000100 01100011 // blt x1, x2, flag3
9   00000000 00000000 00000000 00010011 // nop
10  00000000 00100001 00010110 01100011 // bne x2, x2, flag4    (flag3)
11  00000000 00010001 00010100 01100011 // bne x2, x1, flag4
12  00000000 00000000 00000000 00010011 // nop
13  00000000 00100001 01100110 01100011 // bltu x2, x2, flag5   (flag4)
14  00000000 00100000 11100100 01100011 // bltu x1, x2, flag5
15  00000000 00000000 00000000 00010011 // nop
16  00000000 00000000 00000000 00010011 // nop                  (flag5)
```

图 4. 3. 7 分支指令验证序列

图 4. 3. 8 预测失败重新取值 PC 变化现象

设计测试方式为将两个相同的分支指令类型并排，前一条为不满足跳转条件不进行跳转而后一条为满足跳转条件进行跳转，并将跳转地址设置为该跳转分支指令的下下一条，那么当该指令依照预测器初始状态按照不跳转进行分支预测时，那么当时下下条指令会在 IF 阶段，而伴随着 EXE 阶段发现分支预测失败而在下个周期进行流水线冲刷，该地址指令又会被重新取指，那么便会出现连续两个周期的下下条指令地址的波形显示，凭借这一现象便能够检验该分支指令是否能够正常工作。

图 4.3.8 前三类分支指令波形验证

图 4.3.9 后二类分支指令波形验证

可以看到，因为每类分支前一条均会因为不满足条件而不进行跳转，而且分支预测也按照不跳转进行，于是并不会发生流水线冲刷并出现重新取指令的情况，而将后一条指令设置为满足跳转条件并将跳转地址设置为该分支指令的下下条，因为分支预测不跳转而正常向下取指，那么将会发生流水线冲刷事件，而恰好当前 IF 阶段指令地址为跳转地址，那么在下一个周期的上升沿会被重新赋为该值，于是出现 PC 保持某一数值两个周期的情况，便能够通过这种情况得知预测不跳转失败，进而检验分支指令的正确性。

5 数据前推相关问题

5.1 数据前推相关问题以及实现

实现原因，译码阶段从寄存器拿出的数据可能并不是最新的，五级流水线指令并行的缘由，因此在指令顺序执行的情况下可能会发生 RAW 冲突，借助数据前推模块我们能够将当前 EXE 阶段以及 MEM 阶段的数值推送至前面作为下一周期 EXE 阶段的输入，考虑到乘除器计算的时序问题，需在上升沿到来之前就将 EXE 阶段输入数值更新至最新状态。

实现方式为：创建一个数据前推的模块，从 decoder 模块获得两个源寄存器 rs1_IDout, rs2_IDout，并从 REGS 中拿出从寄存器中获取的数据 data1_REGSout，data2_REGSout， EXE 阶段输入 wb_EXEout 信号以及 rd_EXEout，alu_result_EXEout，MEM 阶段输入 wb_MEMout、ld_MEMout 信号以及 rd_MEMout，alu_result_MEMout 和 从 RAM 中输入 dataRAMout。

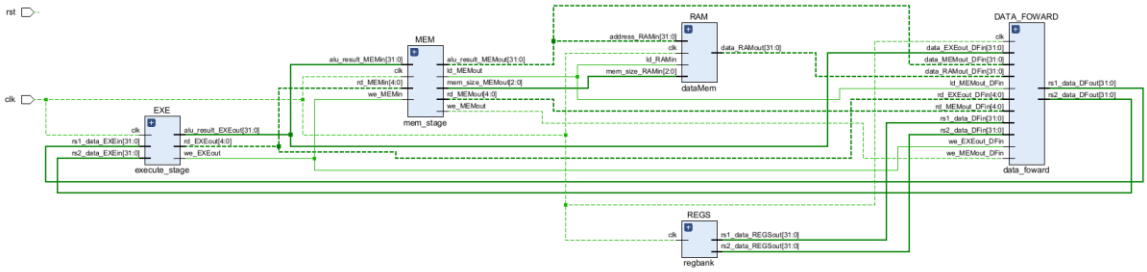


图 5.1 数据前推通路

通过对应冲突以及信号，根据数值更新的优先级，优先使用 EXE 阶段前推的数据，其次使用 MEM 前推的数据。

5.2数据前推上版测试

对一组测试指令进行评测

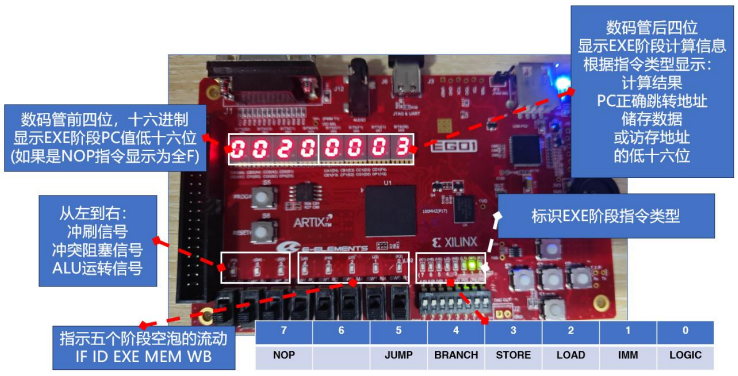


图 5.2.1 板上信息相关

数据前推上机演示：第三条指令到第六条指令的数据前推演示

图 5.2.2 数据前推上机演示

数据前推上板测试详解

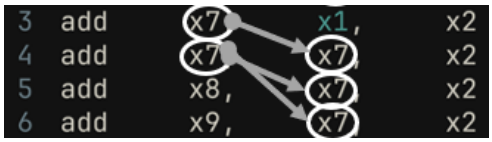


图 5.2.3 数据前推测试指令序列

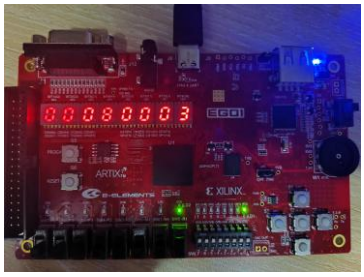


图 5.2.4 x1+x2=>x7

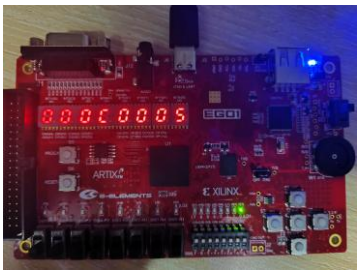


图 5.2.5 x7+x2=>x7（使用前推，dst=1）

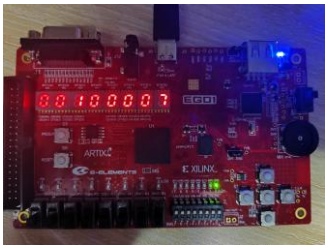


图 5.2.6 x7+x2=>x8（使用前推，dst=1）

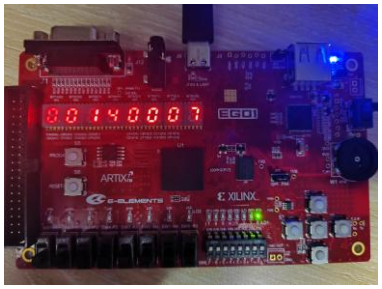


图 5.2.7 x7+x2=>x9（使用前推，dst=2）

5.3数据前推优化分析

数据前推带来的性能提升：使用数据前推，便能够将数据直接前推进行使用无需插入空泡阻塞流水线（除开 Load 引起的指令冲突，需插入一个空泡）。

普通情况：不使用任何优化措施，遇到指令冲突序列，需要在前一条指令 WB 后才能够让接下来的冲突指令执行译码阶段，需要在流水线中插入三个空泡。

使用数据前推：只需要在 load 冲突指令序列的情况塞入一个空泡即可。

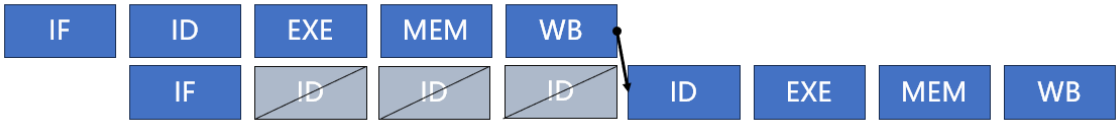


图 5.3.1 未采用数据前推情况

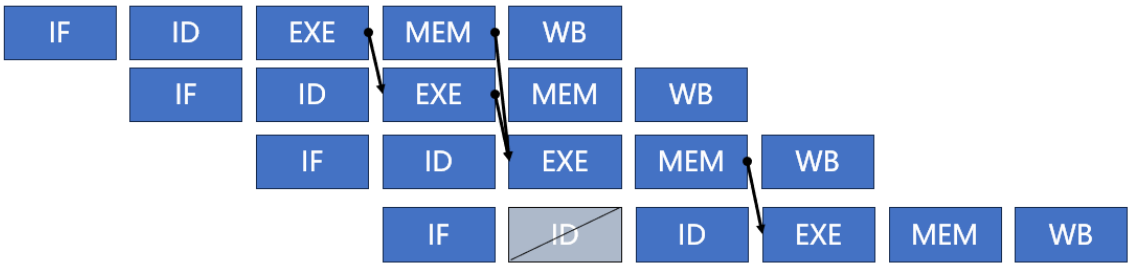


图 5.3.2 采用数据前推情况

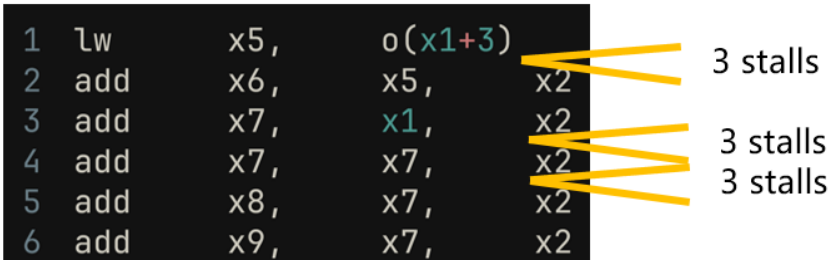


图 5.3.3 数据前推优化前

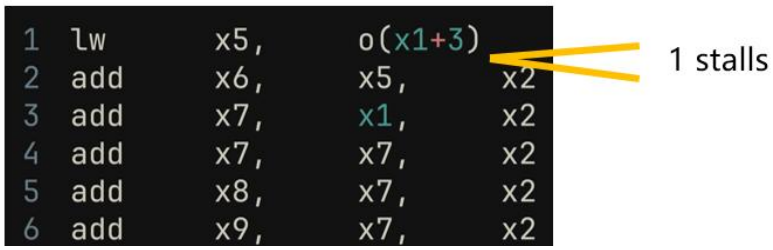


图 5.3.4 数据前推优化后

6 Load 冒险相关问题

6.1Load 冒险相关问题以及实现

使用数据前推可以使得将并未 WB 的数据提前推送至前面供 EXE 阶段的使用，但是依旧无法克服 Load 指令带来的冲突。

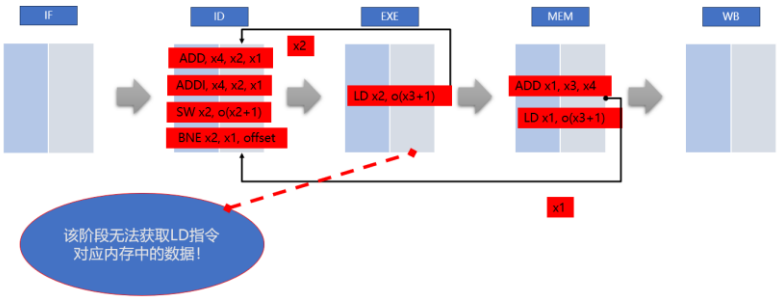


图 6.1.1 Load 指令冲突例子

倘若 EXE 阶段为 load 指令，而 ID 阶段存在数据冒险的情况，没有办法通过数据前推进行解决，因为当 Load 指令还在 EXE 阶段时，无法获取对应地址中的数据，而必须等待该指令流动至 MEM 才能够将数据前推进行使用。

解决方案，当检测到 load 指令冲突的情况，在该对冲突指令之间塞入一个空泡，使得 EXE 阶段放置一个空泡，当 load 指令在 MEM 阶段时，与其发生冲突的指令还停留在 ID 阶段，如此便能够完成数据前推的操作。

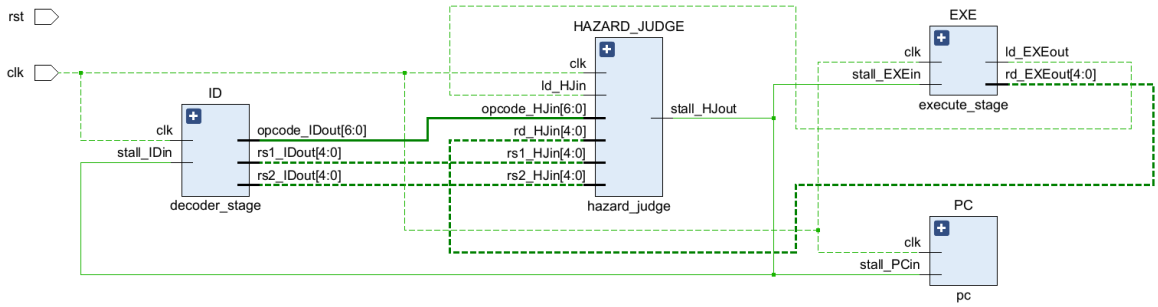


图 6.1.2 load 冲突判断模块

实现方法：假设当前 EXE 阶段为 load 指令，将 EXE 阶段的 ld_EXEout，rd_EXEout 等数据流入到 HAZARD_JUDGE 判断模块，并检测当前 ID 阶段的指令是否与 load 指令发生冲突，倘若发生冲突，将阻塞信号传递给 PC、ID、以及 EXE，

PC 和 ID 暂停流水处理，并使用上一阶段的缓存维持当前状态，将 EXE 的输入替换为空指令的输入状态，以此达到在 EXE 阶段塞入一个 NOP 指令的目的，并保证前两个阶段的状态不发生改变。

6.2冒险相关问题上板演示

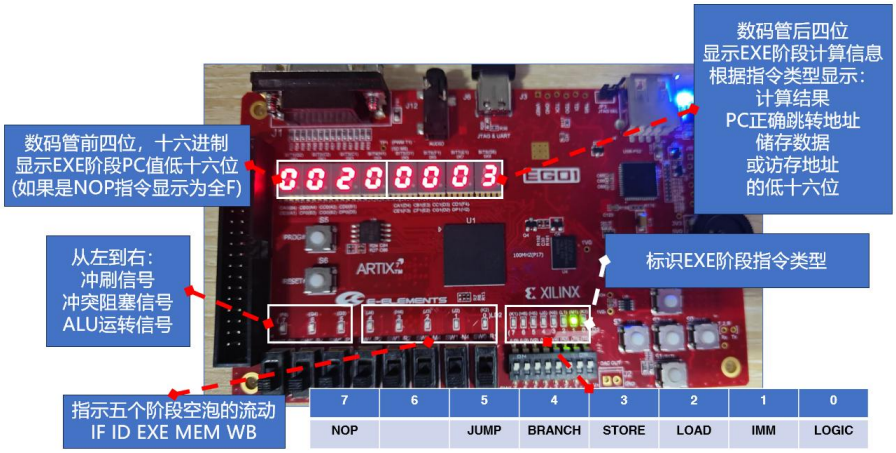


图 6.2.1 板上信息相关

图 6.2.2 Load 指令冲突序列

第一条指令和第二条指令之间发生冲突的上机演示

图 6.2.3 Load 指令冲突序列解决方案上机演示

指令冒险现象详解

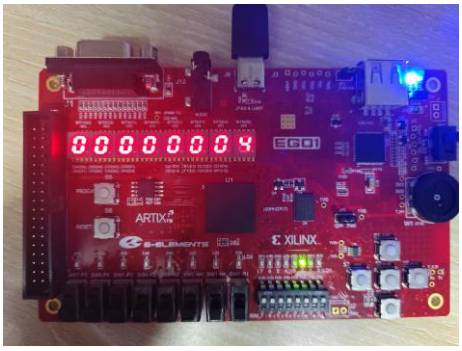


图 6.2.4 Load 指令，以及访存地址显示

此时 load 指令在 EXE 阶段，add 指令在 ID 阶段，检测到这两条指令会发生冲突，那么更新 stall 信号，并在下一个上升沿执行阻塞前两个阶段并在 EXE 阶段插入 NOP 的操作。

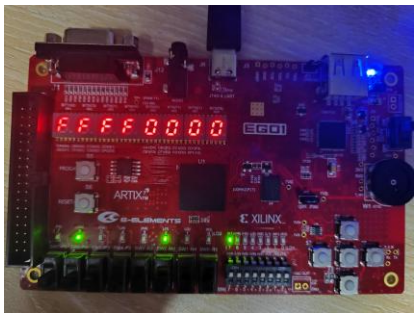


图 6.2.5 EXE 阶段插入空泡显示

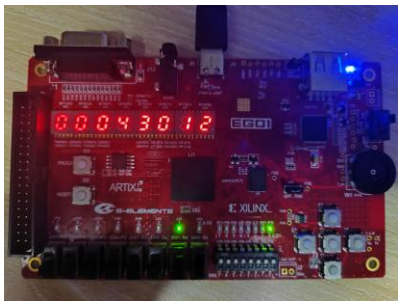


图 6.2.6 Add 指令流入 EXE，并使用数据前推工作

当与 load 指令发生冲突的指令，在两条指令之间插入一个空泡之后，当 load 指令流动到 MEM 阶段，该 ADD 指令还停留在 ID 阶段，那么便可以使用数据前推完成计算。

7 分支预测相关问题以及实现

7.1分支预测原理及其实现

我们可以对过往分支跳转情况进行储存，当该分支跳转指令重复出现时，可以通过查表对该分支的跳转情况进行动态分支预测。

实现方法：2-bit 预测器以及全局地址跳转表，对过往存在的跳转分支进行表项存储，每一表项包含一个 2bit 位以及跳转地址，设置表大小为 64，选取 pc[7:2]位作为索引值。

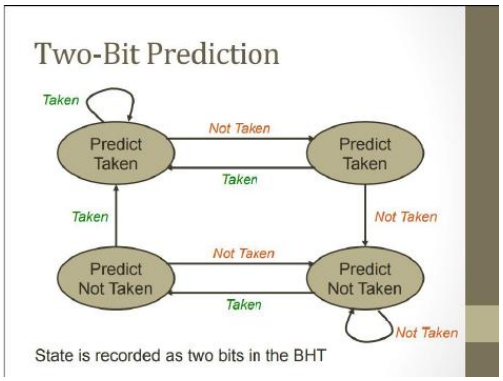


图 7.1.1　2bit 状态位转变

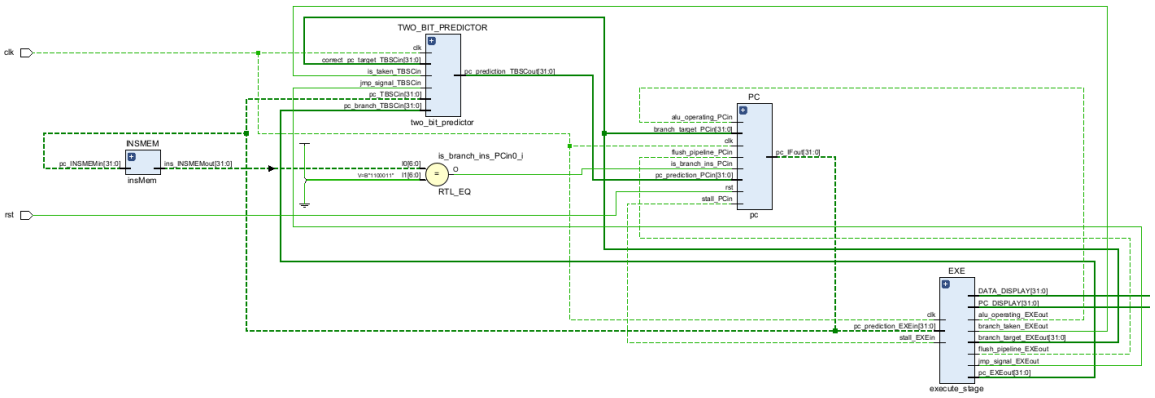


图 7.1.2 分支预测通路图

通过过往分支指令的跳转结果，存放之全局历史表中，每当遇到相同的 pc 索引值，那么根据预测器对应表项完成分支预测，每个表项包含 2bit 状态，以及预测跳转地址，倘若该表项的 2bit 预测值的大小是>=weak_taken 那么将预测跳转地址作为下一个取指令的地址，以此来达到对分支指令的预测效果。

7.2分支预测失败解决方案

倘若分支预测失败，那么意味着我们取出了错误的指令，通过 EXE 阶段分支指令的计算结果，我们能够发现当前 IF 阶段以及 ID 阶段中停留的为错误的指令，这两个指令也是不应该得到执行的，于是当我们发现分支预测失败的时候就将这两条指令替换为 NOP 指令，也即向流水线中插入两个空泡。

实现方法：由 EXE 阶段向 IF 阶段和 ID 阶段发送 flush 信号（该 flush 信号会在 EXE 的下降沿根据该阶段的状态进行更新），并在该 EXE 阶段内部设置用于储存 flush 状态的寄存器，同时 EXE 阶段向 IF 阶段发送正确的跳转地址，用于设置正确的取值地址。

IF 阶段遇到 flush 信号时，将取指 pc 值设置为正确分支跳转地址。

ID 阶段遇到 flush 信号时，将指令状态更新为 NOP

EXE 阶段遇到 flush 信号时，将该阶段的状态更新为 NOP 相关的输入状态。

7.3分支预测失败现象上板

```
add     x7,     x2,     x3
bge     x4,     x2,     offset(-2)
add     x7,     x2,     x3
add     x7,     x2,     x3
```

图 7.3.1 上机测试指令

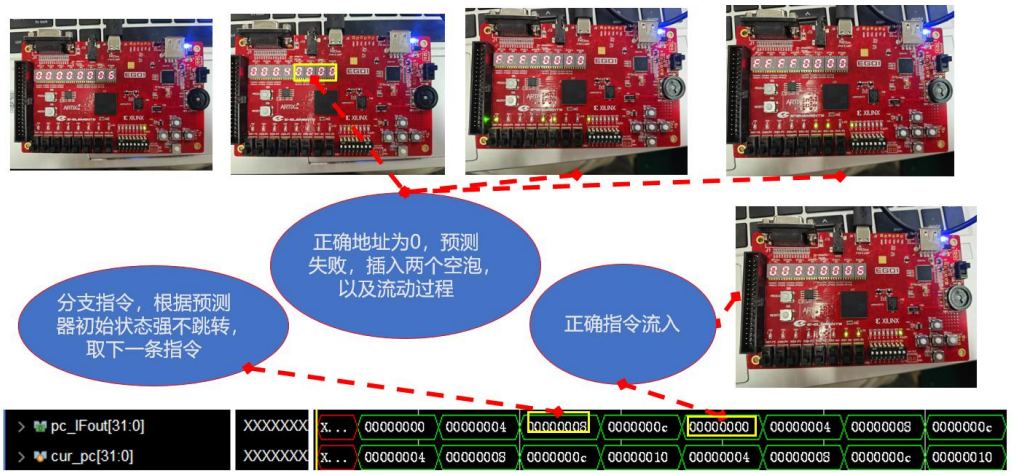


图 7.3.2 分支预测上机测试

分支预测上板测试详解

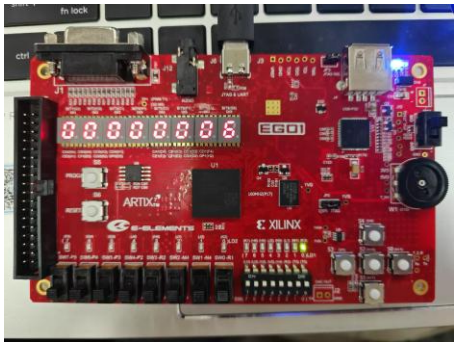


图 7.3.3 流入第一条加法指令

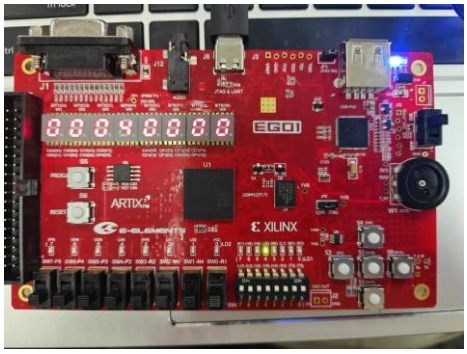


图 7.3.4 分支指令显示跳转地址为 0，预测失败插入空泡

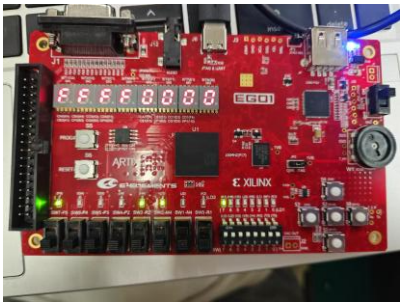


图 7.3.5 第一个空泡流入

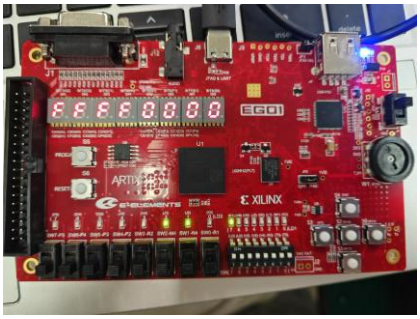


图 7.3.6 第二个空泡流入

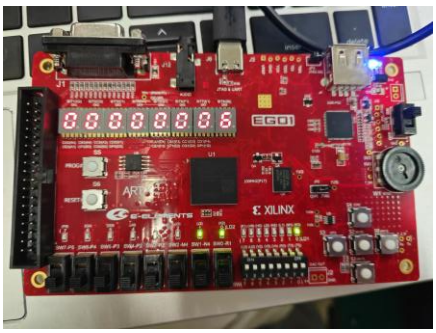


图 7.3.7 正确地址流入

图 7.3.8 IF 阶段 PC 值变化

当 pc_IFout 为 0000004 时表示该指令为分支指令，根据 2bit 的默认状态（强不跳转），该分支会采取不跳转的选择，那么下一条指令将会从 pc+4 的地址正常取出指令，也即 00000008，当 IF 连续取出两条错误指令后冲刷管道并向正确地址取出正确指令，那是因为该分支指令经过两个周期后流动到 EXE 阶段，并验证该分支指令的跳转结果，发现预测失败，冲刷管道并将 IF 阶段的取指 pc 设置为正确值。

7.4分支预测优化分析

```
add     x2,     x0,     x0
add     x3,     x0,     x0
bge     x1,     x1,     offset(1)
add     x2,     x1,     x2
bne     x1,     x1,     offset(1)
add     x3,     x1,     x3
bge     x4,     x3,     offset(-5)
```

图 7.4.1 分支预测测试指令

一个 9 层循环中有两条分支指令一个跳转一个不跳转，共冲刷 5 次流水线。27(2*9+9)次分支判断。

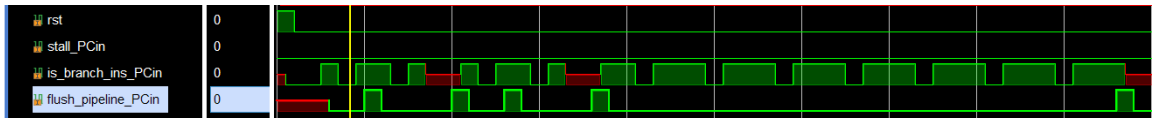


图 7.4.2 分支预测模拟冲刷信号显示

预测方式	预测成功率
2bit	81%
静态跳转	62%
静态不跳转	37%

图 7.4.3 预测准确度分析以及性能提升评估

每个过往分支指令的 pc 状态会被记录，那么对于同一条分支指令地址，他的 2bit 状态位至少需要两个才能够从强不跳转转变为弱跳转，对于第一条分支指令，他始终是跳转的，那么初始的强不跳转会产生两次分支预测错误，对于第二条分支指令始终不跳转那么预测一直正确，而对于最后一条向前跳转分支指令，前八次均为向后跳转，最开始两个预测不跳转错误，以及最后一次预测跳转错误，总计预测失败 5 次，冲刷五次流水线。该情况下动态分支预测成功率大于静态预测。

8 总结

实际在设计五级流水线 cpu 的过程中走了许多弯路，甚至最开始一些天设计的流水结构都推倒重来了，后来为了捋清各模块合适的检测和工作时刻，尝试在纸上对具体的一些指令进行分析，通过不同时刻每个阶段不同状态位的关系，渐渐明白不同检测模块的工作时机，以及一些数据处理的最佳时刻。

为了更好地设计流水结构，回顾了课上的一些课件还有具体的例子，更加深入地理解了不同模块工作时的作用，并按照自己的设计进行了一步一步的验证，最终成功做出了能够适配五级流水线的相关解决模块。

为了更好的呈现上板效果，又更加深入的学习了 ego1 的使用以及 vivado 软件与硬件之间的关联方法，让我对硬件方面有了更加深刻的学习和理解。

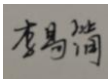
这门课程贯彻数据结构，计算机组成原理以及计算机体系结构，借助该系统综合设计给了我一次回顾过往学习的机会，并同时给出综合性的挑战，让我能够结合不同方面去进行硬件设计方面的考量，我觉得这门课程对我个人而言是一次宝贵的挑战，并让我在硬件方面有了更加深刻的理解以及学习，并挑战以及结合自己学习过的知识将其转化为真实的实践成果。

同时编写报告的过程中，让我对自己做出来的产品进行综合地阐述，能够更加充分的理解自己制作出的产品的思路，让人受益匪浅。

一、原创性声明

本人郑重声明本报告内容，是由作者本人独立完成的。有关观点、方法、数据和文献等的引用已在文中指出。除文中已注明引用的内容外，本报告不包含任何其他个人或集体已经公开发表的作品成果，不存在剽窃、抄袭行为。

特此声明！

作者签字: 

	评价内容	权重	得分
验收		0.4	
设计报告		0.6	
合计			

指导教师（签章）：

2025 年 1 月 10 日